THANDEV - Comprehensive Project Analysis

Generated: 2025-11-23 14:19:44
Status: MVP / Prototype in Active Development
Version: 1.0.0
Author: Ashwathaman Satish Kumar

Executive Summary

Thandev is an **early-stage cybersecurity intelligence platform** that provides real-time CVE (Common Vulnerabilities and Exposures) data, security scanning capabilities, and performance analysis tools. The platform is currently in **active development** with core features implemented and deployed to production environments.

Current State: - ✅ Backend API deployed on Fly.io - ✅ Frontend deployed on Vercel - ✅ Cloud Run services deployed for scanning - ✅ Database migrations configured - ⚠️ Some features are experimental or partially implemented - 📋 Comprehensive testing and monitoring still in roadmap

Key Capabilities: - Real-time CVE data ingestion from NVD - Three-tier security scanning (surface, discovery, active) - Performance scanning with Google Lighthouse (all 4 categories) - News feed aggregation from trusted security sources - Exploit database viewer - GitHub OAuth authentication

---

Project Architecture

Deployment Architecture

Frontend (Vercel)

- **Platform:** Vercel

- **URL:** https://thandev.vercel.app
- **Framework:** Next.js 14 with App Router
- **Language:** TypeScript
- **Status:** ✅ Deployed and operational
- **Features:**
- React components with TypeScript
- NextAuth.js for GitHub OAuth
- Performance scanner UI
- Security scanner UI
- News feed aggregation
- Exploit database viewer
- Responsive design with dark/light themes

Backend API (Fly.io)

- **Platform:** Fly.io
- **URL:** https://thandev-backend.fly.dev
- **Framework:** FastAPI (Python 3.11+)
- **Database:** PostgreSQL (Neon)
- **Status:** ✅ Deployed and operational
- **Features:**
- RESTful API endpoints
- NVD CVE data ingestion
- Security scanning orchestration
- Performance scanning coordination
- Database migrations (Alembic)
- Rate limiting and CORS
- Error tracking (Sentry-ready)

Cloud Run Services (Google Cloud Platform)

Lighthouse Scanner: - **URL:** https://lighthouse-scanner-697855159499.us-central1.run.app - **Status:** ✅ Deployed (revision lighthouse-scanner-00005-xbw) - **Configuration:** - Memory: 2Gi - CPU: 2 cores - Timeout: 900s (15 minutes) -

Capabilities: - Performance scanning - Accessibility scanning - Best Practices scanning - SEO scanning - Core Web Vitals metrics - Optimization opportunities - Diagnostic information

Security Scanner: - **Status:** ✅ Deployed - **Tools Integrated:** - Nuclei (vulnerability scanner) - Nmap (network scanner) - SSLScan (SSL/TLS scanner) - Subfinder (subdomain discovery) - FFuF (web fuzzer) - crt.sh (certificate transparency) - Nikto (web server scanner)

Technology Stack

Backend Stack

Currently Implemented & Used: - **FastAPI** - High-performance async web framework - **SQLAlchemy** - ORM for database operations - **PostgreSQL** - Primary database (via Neon) - **Alembic** - Database migration tool - **httpx** - Async HTTP client for Cloud Run integration - **Pydantic** - Data validation and settings management

Implemented but Experimental: - **Celery** - Background task framework (configured, but tasks are stubs) - **Redis** - Caching (configured, but falls back to in-memory cache)

Planned / Not Yet Integrated: - Comprehensive test suite - Advanced monitoring (beyond Sentry) - CI/CD pipeline

Frontend Stack

Currently Implemented & Used: - **Next.js 14** - React framework with App Router - **TypeScript** - Type-safe development - **Tailwind CSS** - Utility-first styling - **NextAuth.js** - Authentication library - **React Hooks** - State management

Planned: - E2E testing (Playwright/Cypress) - Performance monitoring - Advanced error boundaries

Cloud Services

Currently Implemented & Used: - **Google Cloud Run** - Serverless container platform - **Neon PostgreSQL** - Serverless PostgreSQL database - **Vercel Edge Network** - CDN and edge functions

---

Feature Implementation Status

✅ Currently Implemented & Used in Code

1. **Authentication System**
2. GitHub OAuth via NextAuth.js
3. JWT-based session management
4. User profile management
5. Status: Fully functional, tested locally
6. **CVE Data Ingestion**
7. NVD API v2.0 integration
8. Automated 6-hour refresh schedule
9. Rate limiting (50 req/30s, 5000 with API key)
10. Data normalization and deduplication
11. Status: Functional, tested with real NVD data
12. **Security Scanning**
13. Three-tier scanning system:
    - Level 0: Surface scan (quick, non-intrusive)
    - Level 1: Discovery scan (port scanning, subdomain discovery)
    - Level 2: Active scan (vulnerability scanning, fuzzing)
14. Cloud Run Jobs integration
15. Result processing and storage
16. Status: Functional, tested locally and in Cloud Run
17. **Performance Scanning**
18. Google Lighthouse integration
19. All 4 categories: Performance, Accessibility, Best Practices, SEO
20. Core Web Vitals metrics
21. Optimization opportunities

22. Diagnostic information
23. Cloud Run Service integration

24. Status: Fully functional, deployed and tested

25. **News Feed Aggregation**

26. RSS feed parsing from trusted sources
27. Image extraction and validation
28. Category filtering (cyber, ai, tech)

29. Status: Functional, tested with real feeds

30. **Exploit Database**

31. CVE-based exploit tracking
32. Search and filtering

33. Status: Functional

34. **Database Management**

35. Alembic migrations
36. Table creation and updates
37. Connection pooling
38. Status: Functional, migrations tested

⚠️ Implemented but Experimental / Partial

1. **Celery Background Tasks**
2. Celery app configured
3. Tasks defined but are stubs (not fully implemented)

4. Status: Infrastructure ready, tasks need implementation

5. **Redis Caching**

6. Redis connection configured
7. Falls back to in-memory cache if Redis unavailable

8. Status: Works but not fully utilized

9. **Multi-tier Security Scan Orchestration**
10. Basic orchestration implemented
11. Could be more comprehensive
12. Status: Functional but could be enhanced

📋 Planned / Not Yet Integrated

1. **CI/CD Pipeline**
2. GitHub Actions workflows
3. Automated testing
4. Deployment automation
5. Status: Not implemented
6. **Comprehensive Test Coverage**
7. Unit tests
8. Integration tests
9. E2E tests
10. Status: Not implemented
11. **Advanced Monitoring & Alerting**
12. Beyond basic Sentry integration
13. Custom dashboards
14. Alert rules
15. Status: Not implemented
16. **User Roles & Permissions**
17. Role-based access control
18. Permission system
19. Status: Not implemented
20. **API Rate Limiting Per User**
21. User-specific rate limits

22. Quota management
23. Status: Not implemented
24. **Webhook Notifications**
25. Scan completion webhooks
26. Status: Not implemented
27. **Scheduled Scans**
28. Recurring scan scheduling
29. Status: Not implemented
30. **Report Generation**
31. PDF/HTML report export
32. Status: Not implemented

What's Working

✅ Fully Functional Components

1. **Backend API Core**
2. FastAPI application runs correctly
3. All routers functional
4. Database connections work
5. Health checks operational
6. Status: Tested locally and in production
7. **Frontend UI**
8. Next.js application functional
9. Authentication works
10. All pages render correctly
11. API integration working

12. Status: Tested locally and in production

13. **Performance Scanning**

14. Cloud Run Service deployed and functional
15. All 4 Lighthouse categories working
16. Results properly formatted and displayed

17. Status: Deployed and tested

18. **Security Scanning**

19. Three-tier system operational
20. Cloud Run Jobs integration working
21. Results stored in database

22. Status: Functional, tested locally

23. **CVE Data Ingestion**

24. NVD API integration working
25. Data normalization functional
26. Deduplication working

27. Status: Tested with real NVD data

28. **Database Operations**

29. Migrations run successfully
30. Tables created correctly
31. Queries functional
32. Status: Tested in production

What Needs Work

🔴 High Priority Improvements

1. **Testing Coverage**

2. **Current State:** No automated tests
3. **Needed:** Unit tests, integration tests, E2E tests
4. **Impact:** High - affects reliability and maintainability
5. **Monitoring & Observability**
6. **Current State:** Basic Sentry integration (if configured)
7. **Needed:** Comprehensive logging, metrics, dashboards
8. **Impact:** High - affects debugging and production reliability
9. **Error Handling**
10. **Current State:** Basic error handling
11. **Needed:** More robust error handling, retry logic, graceful degradation
12. **Impact:** Medium - affects user experience
13. **Database Optimization**
14. **Current State:** Basic indexes, some JSONB usage
15. **Needed:** Query optimization, additional indexes, connection pooling tuning
16. **Impact:** Medium - affects performance at scale
17. **CI/CD Pipeline**
18. **Current State:** Manual deployments
19. **Needed:** Automated testing, deployment pipelines
20. **Impact:** Medium - affects development velocity

🟡 Medium Priority Improvements

1. **Celery Tasks Implementation**
2. **Current State:** Tasks are stubs
3. **Needed:** Full implementation of background tasks
4. **Impact:** Medium - affects scalability
5. **Redis Caching Strategy**

6. **Current State:** Basic fallback to in-memory
7. **Needed:** Comprehensive caching strategy
8. **Impact:** Medium - affects performance
9. **Documentation**
10. **Current State:** Basic README, some deployment docs
11. **Needed:** API documentation, architecture docs, runbooks
12. **Impact:** Low - affects developer onboarding
13. **Security Hardening**
14. **Current State:** Basic security measures
15. **Needed:** Security audit, penetration testing, advanced security features
16. **Impact:** High - affects security posture

Recent Fixes & Improvements

November 2025

1. **Fixed Cloud Run Service to scan all 4 Lighthouse categories**
2. Previously only scanned performance
3. Now scans: performance, accessibility, best-practices, seo
4. Deployment: lighthouse-scanner-00005-xbw
5. **Fixed fake/NaN scores in frontend**
6. Added validation for missing/invalid data
7. Shows 'N/A' instead of NaN or fake values
8. Better error handling
9. **Fixed Cloud Run environment variable handling**
10. Strips incorrect CLI flags from environment variables
11. Better error messages

12. **Fixed database table creation**
13. Corrected imports in init_db.py
14. Integrated Alembic migrations into Dockerfile
15. Ensures all tables are created on deployment
16. **Fixed performance scan to use Cloud Run instead of local execution**
17. Prevents OOM errors in backend
18. Better scalability
19. Proper async HTTP calls
20. **Fixed frontend-backend API communication**
21. Corrected environment variable usage
22. Fixed redirect issues
23. Better error handling
24. **Fixed NextAuth configuration**
25. Support for both GITHUB_ID/GITHUB_SECRET and GITHUB_CLIENT_ID/GITHUB_CLIENT_SECRET
26. Removed invalid trustHost option
27. **Fixed ESLint version conflicts**
28. Pinned eslint-config-next to v14.2.5
29. Prevents upgrade to v16 which requires ESLint 9

Metrics & Capacity

Design Goals (Not Yet Load Tested)

Note: The following are design goals and intended capabilities. They have not been load tested or verified at scale.

- **CVE Records:** Designed to handle large CVE datasets (tested with sample data)
- **Concurrent Users:** Architecture designed to scale (not yet load tested)
- **Scan Durations:**
- Performance scans: ~60-90 seconds (scanning 4 categories)
- Security scans: 5-60 minutes depending on tier
- **Rate Limiting:** 50 requests/30 seconds (5000 with API key)

Actual Tested Metrics

- **Local Development:** All features tested locally
- **Production Deployment:** Backend and frontend deployed and operational
- **Cloud Run Services:** Deployed and functional
- **Database:** Migrations tested, tables created successfully

---

Security & Reliability

Current Security Measures

- **Authentication:** GitHub OAuth via NextAuth.js
- **CORS:** Configured for frontend domain
- **Rate Limiting:** Basic rate limiting implemented
- **Input Validation:** Pydantic models for API validation
- **SQL Injection Prevention:** SQLAlchemy ORM protection
- **Error Handling:** Secure error responses (no sensitive data leaked)

Security Architecture

- **Designed to be multi-tenant friendly** (single-tenant implementation for now)
- **Core auth and basic protections implemented**
- **Advanced hardening and monitoring are part of the roadmap**

Reliability

- **Core functionality tested locally**
- **Production deployments operational**
- **Error tracking configured (Sentry-ready)**
- **Database migrations tested**
- **Comprehensive monitoring and alerting still in roadmap**

Overall Assessment

Status: Early-stage MVP with a production-style architecture and clear roadmap for hardening.

Strengths: - ✅ Core features implemented and functional - ✅ Modern tech stack - ✅ Proper deployment architecture - ✅ Database migrations configured - ✅ Cloud Run integration working - ✅ Recent fixes address major issues

Areas for Improvement: - ⚠️ Testing coverage needed - ⚠️ Monitoring and observability needs enhancement - ⚠️ Some features are experimental or partial - ⚠️ Documentation could be more comprehensive

Roadmap: - 📋 Comprehensive testing - 📋 Advanced monitoring - 📋 CI/CD pipeline - 📋 Security hardening - 📋 Performance optimization - 📋 Feature enhancements

---

Author's Reflection

This project represents an early-stage MVP of a cybersecurity intelligence platform. The architecture is designed for scalability and production use, but many components are still in active development.

What was hard: - Integrating multiple Cloud Run services - Handling environment variable configuration across platforms - Debugging deployment issues (Fly.io, Vercel, Cloud Run) - Ensuring proper data flow between frontend, backend, and Cloud Run services

What broke in deployment: - Environment variable handling (CLI flags getting into values) - Database table creation (missing imports) - Performance scan OOM errors (fixed by moving to Cloud Run) - Frontend-backend API communication (redirect issues) - NaN scores in frontend (fixed with validation)

What I learned: - FastAPI async patterns and Cloud Run integration - Next.js App Router and API routes - Google Cloud Run Services vs Jobs - Database migrations with Alembic - Environment variable management across platforms - Error handling and validation in production

Current State: The platform is functional for basic use cases. Core features work, but comprehensive testing, monitoring, and advanced features are still in development. The architecture is solid and ready for scaling once these improvements are made.

Document Generated: 2025-11-23 14:19:44
Project Version: 1.0.0
Status: MVP / Prototype in Active Development

This document reflects the state of the project as of 2025-11-23. Some parts may lag behind the current codebase as development continues.